a1_6681224

Instruction : According to the AirVisual mobile application as show in the figure 1, you must design the diagram 1.1-1.2 (low level design for programmers). The color code of AQI level will be updated according to AQI ranks in figure 2. The application will send the current location as city name to retrieve the latest information including AQI, temperature, wind speed, and humidity from the AirVisual webservice as a backend API and this webservice queries the latest data from AirVisual database server. Note. The webservice API and database servers are located in different servers.

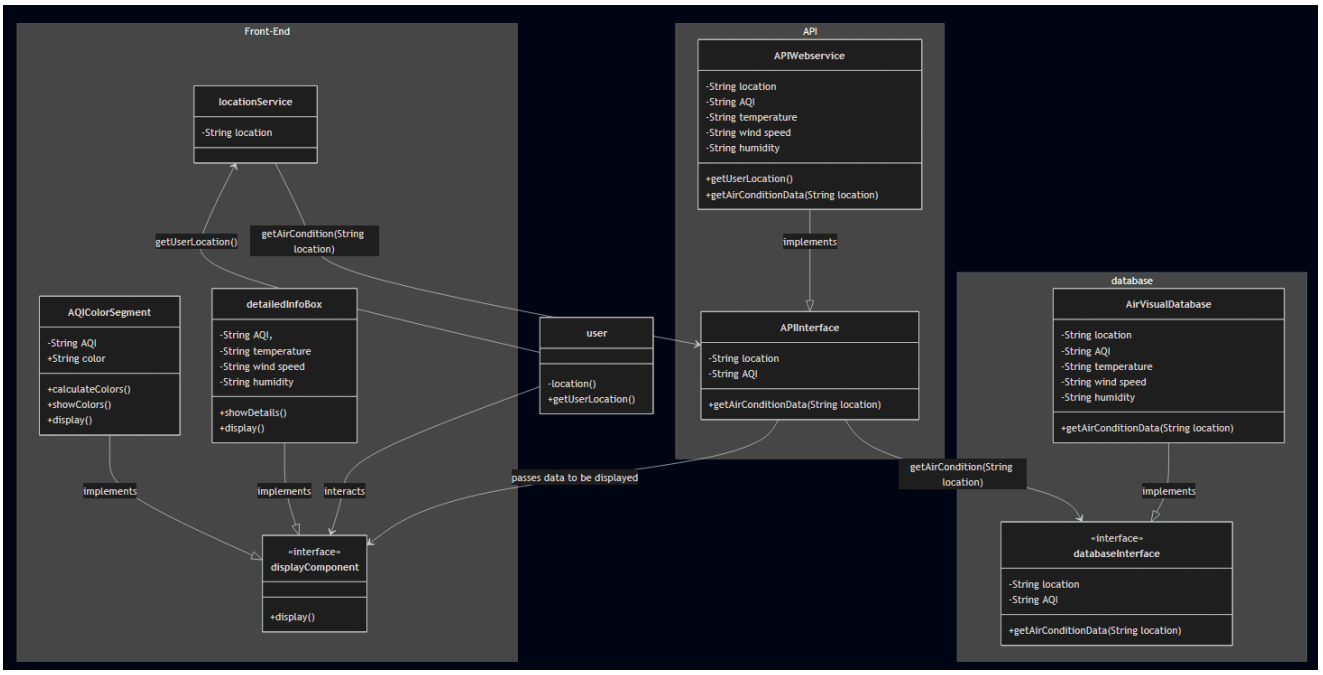
Class diagram

```
%% - The ==color code== of AQI level will be

%% ==updated== according to AQI ==ranks== in figure 2.

%% - The application will send the

%% ==current location== as ==city name== to retrieve

%% the latest information including

%%  - AQI,

%%  - temperature,

%%  - wind speed, and

%%  - humidity

%%  - from the AirVisual webservice

%% - ==AirVisual webservice== as a ==backend API==

%% and this webservice queries the latest data from

%% ==AirVisual database server==. Note.

%% The webservice API and database servers are located in

%% different servers.

classDiagram

    class user{
```

```
        -location()
        +getUserLocation()
    }
    namespace Front-End{
        class displayComponent{
            <<interface>>
            +display()
        }
        class AQIColorSegment{
            -String AQI
            +String color
            +calculateColors()
            +showColors()
            +display()
        }
        class detailedInfoBox{
            -String AQI,
            -String temperature
            -String wind speed
            -String humidity
            +showDetails()
            +display()
        }
        class locationService{
            -String location
```

```
    }
}
namespace API{
    class APIInterface{
        -String location
        -String AQI
        +getAirConditionData(String location)
    }
    class APIWebservice{
        -String location
        -String AQI
        -String temperature
        -String wind speed
        -String humidity
        +getUserLocation()
        +getAirConditionData(String location)
    }
}
namespace database{
    class databaseInterface{
        <<interface>>
        -String location
        -String AQI
        +getAirConditionData(String location)
    }
```

```
        class AirVisualDatabase{

            -String location

            -String AQI

            -String temperature

            -String wind speed

            -String humidity

            +getAirConditionData(String location)

        }

    }



    AQIColorSegment --|> displayComponent : implements

    detailedInfoBox --|> displayComponent : implements

    user --> displayComponent : interacts

    locationService <-- user : getUserLocation()

    locationService --> APIInterface : getAirCondition(String location)

    APIWebservice --|> APIInterface : implements

    APIInterface --> databaseInterface : getAirCondition(String location)

    AirVisualDatabase --|> databaseInterface : implements

    APIInterface --> displayComponent : passes data to be displayed
```

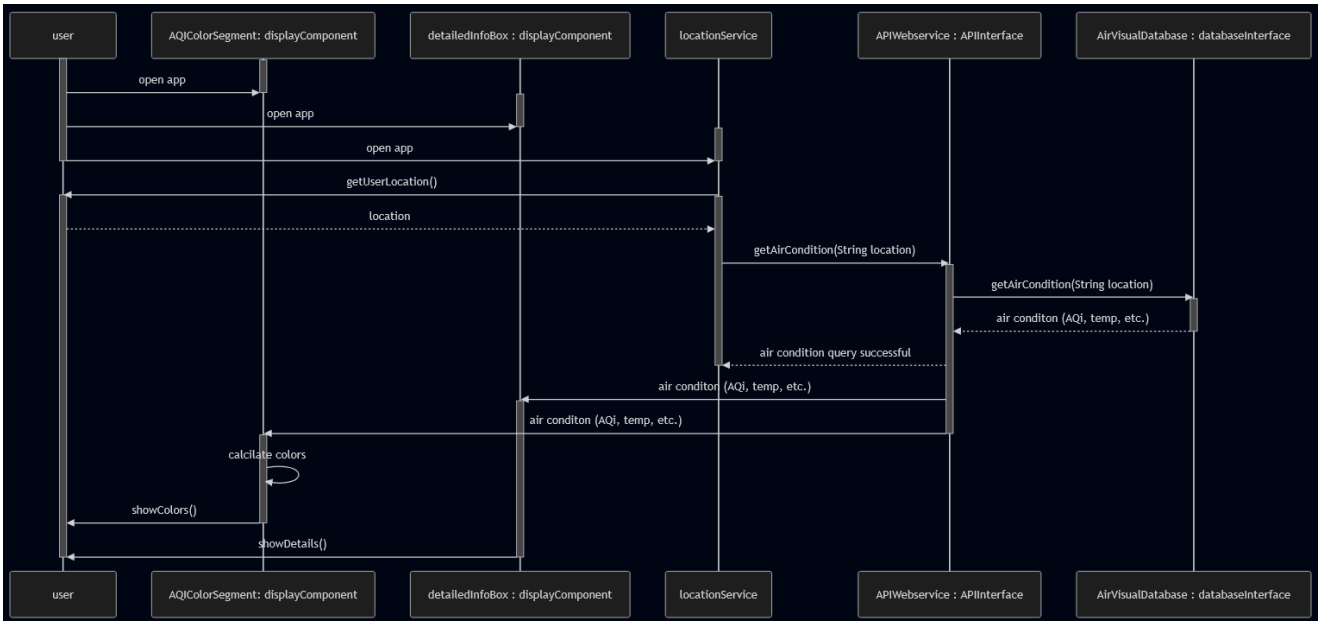


Sequence diagram

```
sequenceDiagram



participant user

%%front end

participant AQIColorSegment as AQIColorSegment: displayComponent

participant detailedInfoBox as detailedInfoBox : displayComponent

participant locationService

%%API

participant APIWebservice as APIWebservice : APIInterface

%%Database

participant AirVisualDatabase as AirVisualDatabase : databaseInterface



activate user

activate AQIColorSegment

user ->> AQIColorSegment : open app
```

```
deactivate AQIColorSegment

activate detailedInfoBox

user ->>detailedInfoBox : open app

deactivate detailedInfoBox

activate locationService

user ->> locationService : open app

deactivate locationService

deactivate user



locationService ->> user : getUserLocation()

activate user

activate locationService

user -->> locationService : location

locationService ->> APIWebservice : getAirCondition(String location)

activate APIWebservice

APIWebservice ->> AirVisualDatabase : getAirCondition(String location)

activate AirVisualDatabase

AirVisualDatabase -->> APIWebservice : air conditon (AQi, temp, etc.)

deactivate AirVisualDatabase

APIWebservice -->> locationService : air condition query successful

deactivate locationService

APIWebservice ->> detailedInfoBox : air conditon (AQi, temp, etc.)

activate detailedInfoBox

APIWebservice ->> AQIColorSegment : air conditon (AQi, temp, etc.)

deactivate APIWebservice
```

```
activate AQIColorSegment

AQIColorSegment ->> AQIColorSegment : calcilate colors

AQIColorSegment ->> user : showColors()

deactivate AQIColorSegment

detailedInfoBox ->> user : showDetails()

deactivate detailedInfoBox

deactivate user
```

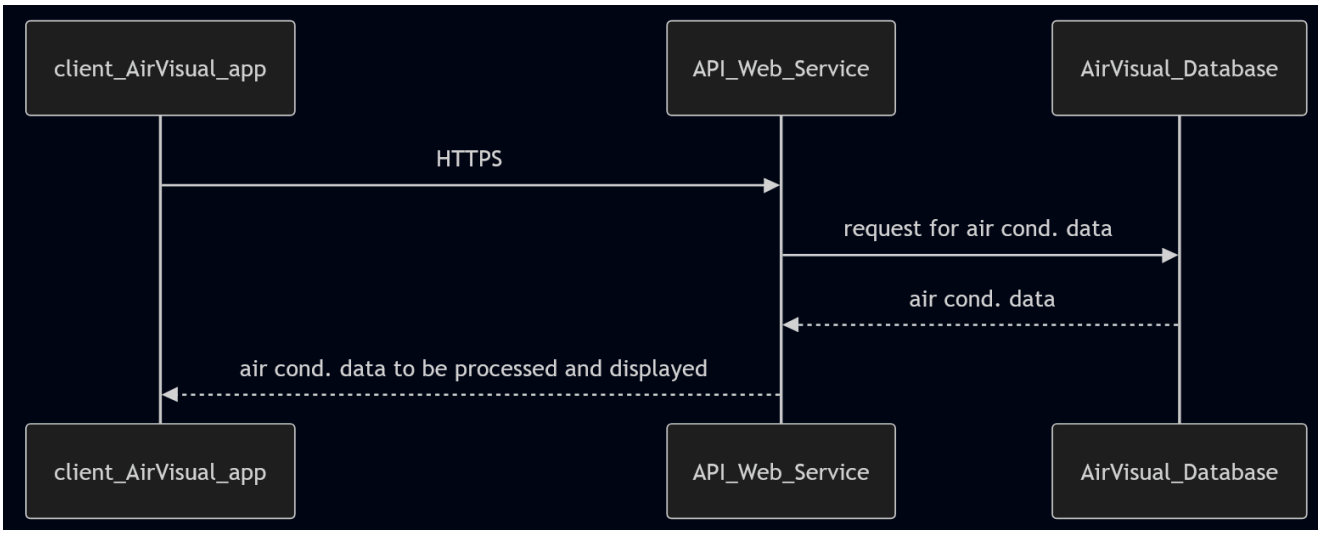


System architecture model

Three tiers architecture

```
sequenceDiagram



participant client_AirVisual_app

participant API_Web_Service

participant AirVisual_Database



client_AirVisual_app ->> API_Web_Service : HTTPS
```

```
API_Web_Service ->> AirVisual_Database : request for air cond. data

AirVisual_Database -->> API_Web_Service : air cond. data

API_Web_Service -->> client_AirVisual_app : air cond. data to be processed
and displayed
```

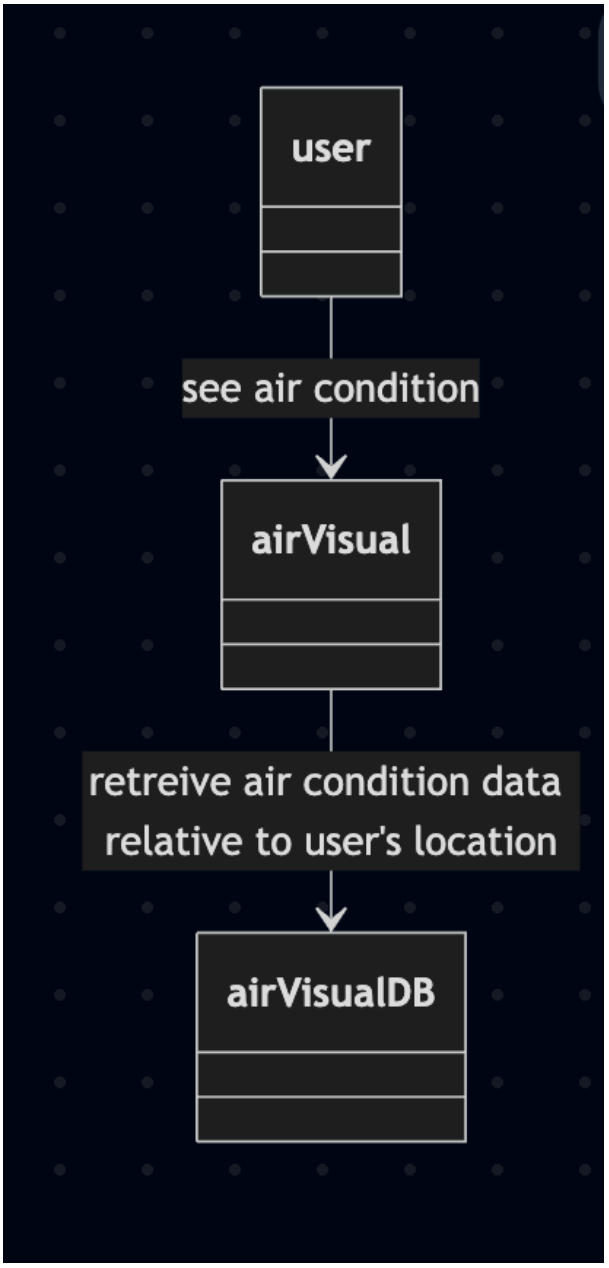


C4 model

1. **Context diagram**

```
classDiagram


class user{


}

class airVisual{


}

class airVisual DB{


}
```

```
user --> airVisual : see air condition

airVisual --> airVisualDB : retreive air condition data relative to user's
location
```

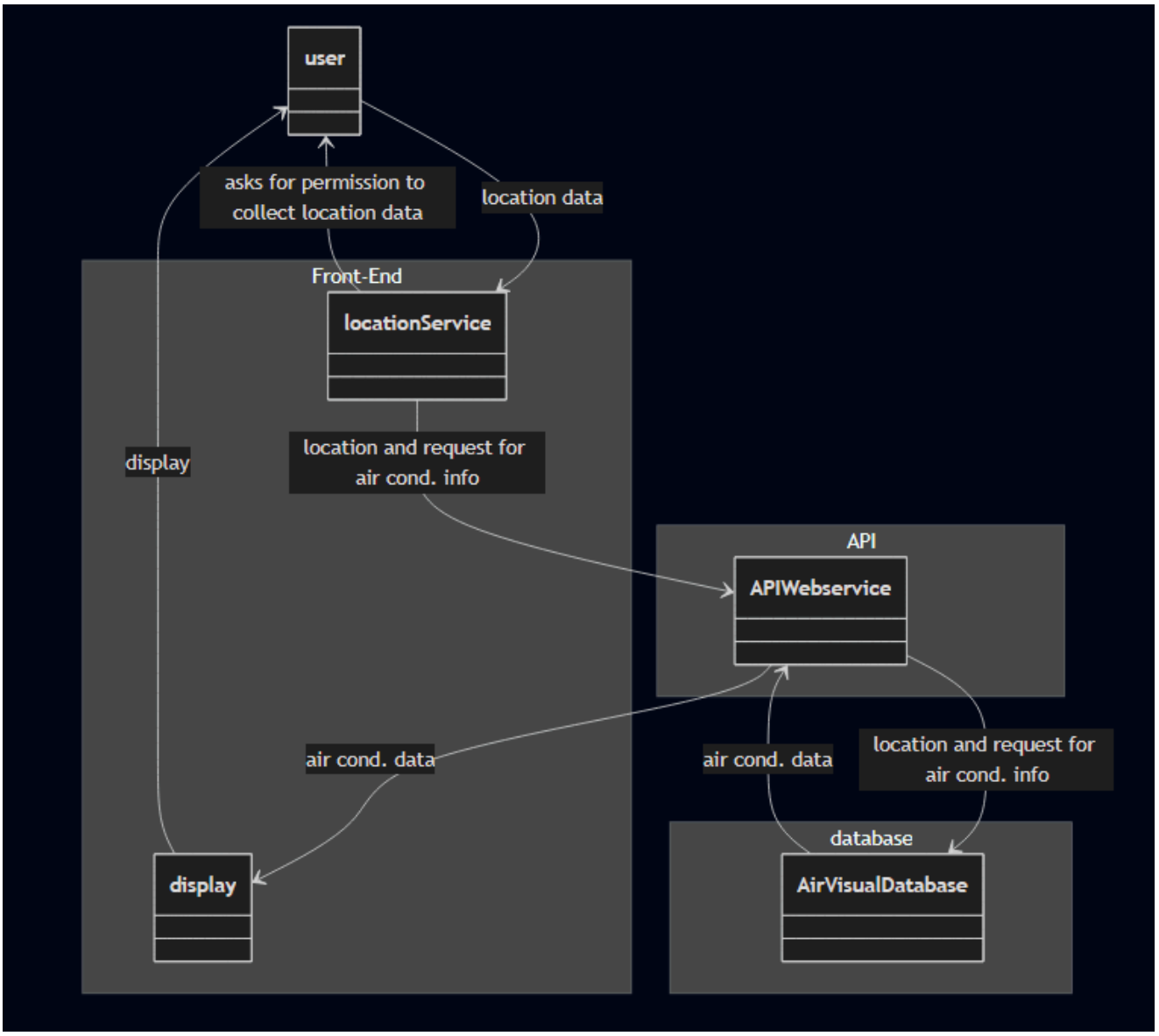


2. **Container diagram**

```
%% - The ==color code== of AQI level will be

%% ==updated== according to AQI ==ranks== in figure 2.

%% - The application will send the

%% ==current location== as ==city name== to retrieve

%% the latest information including

%%  - AQI,
```

```
%%  - temperature,
%%  - wind speed, and
%%  - humidity
%%  - from the AirVisual webservice
%% - ==AirVisual webservice== as a ==backend API==
%% and this webservice queries the latest data from
%% ==AirVisual database server==. Note.
%% The webservice API and database servers are located in
%% different servers.
classDiagram
    class user{
    }
    namespace Front-End{
        class display{


        }
        class locationService{
        }
    }
    namespace API{
        class APIWebservice{
        }
    }
    namespace database{
        class AirVisualDatabase{
```

```
        }

    }



locationService --> user : asks for permission to collect location data

user --> locationService : location data

locationService --> APIWebservice : location and request for air cond. info

APIWebservice --> AirVisualDatabase : location and request for air cond.
info

AirVisualDatabase --> APIWebservice : air cond. data

APIWebservice --> display : air cond. data

display --> user : display
```

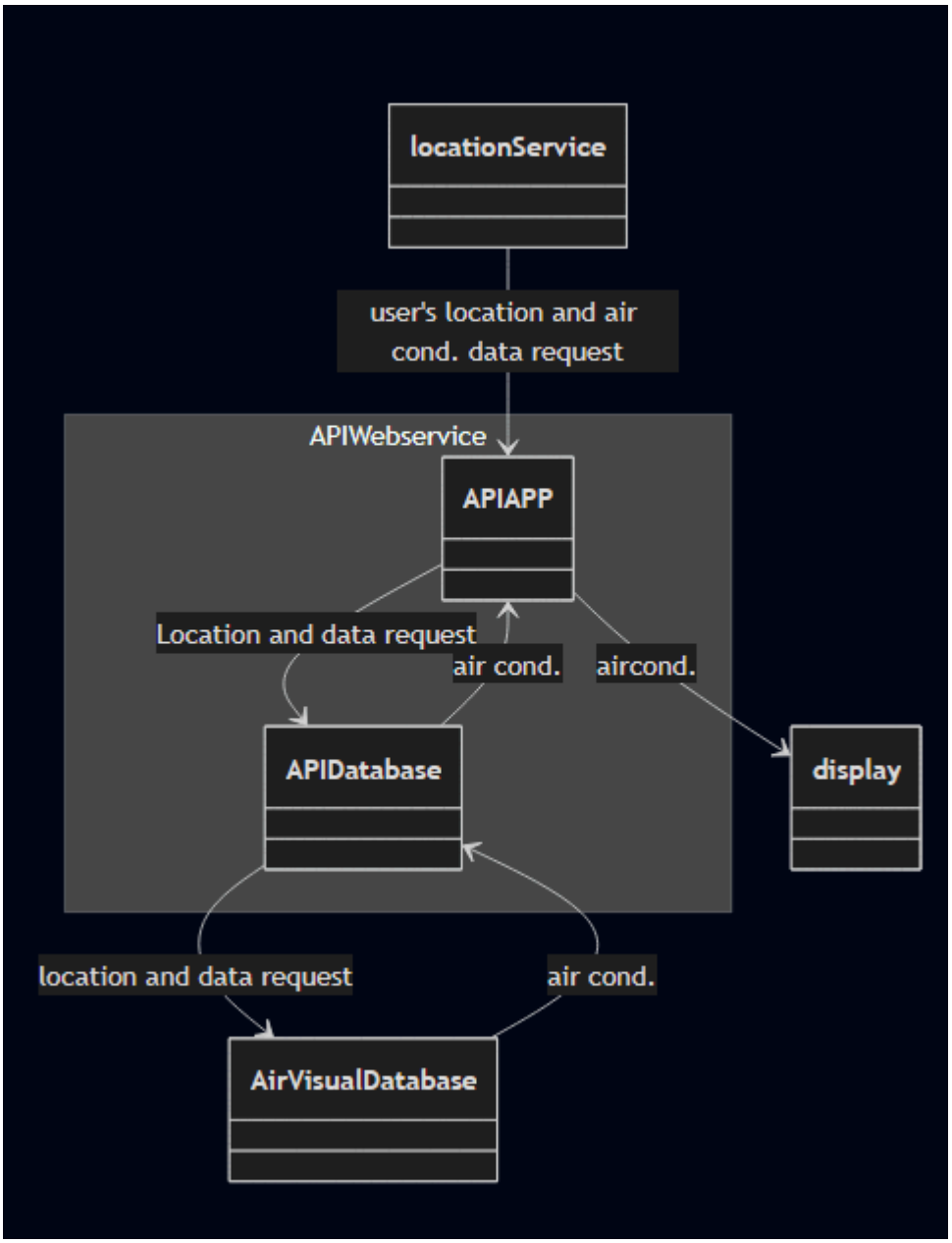


3. **Component diagram**

APIWebservice component diagram

```
classDiagram

namespace APIWebservice{

    class APIAPP{


    }

    class APIDatabase{


    }
```

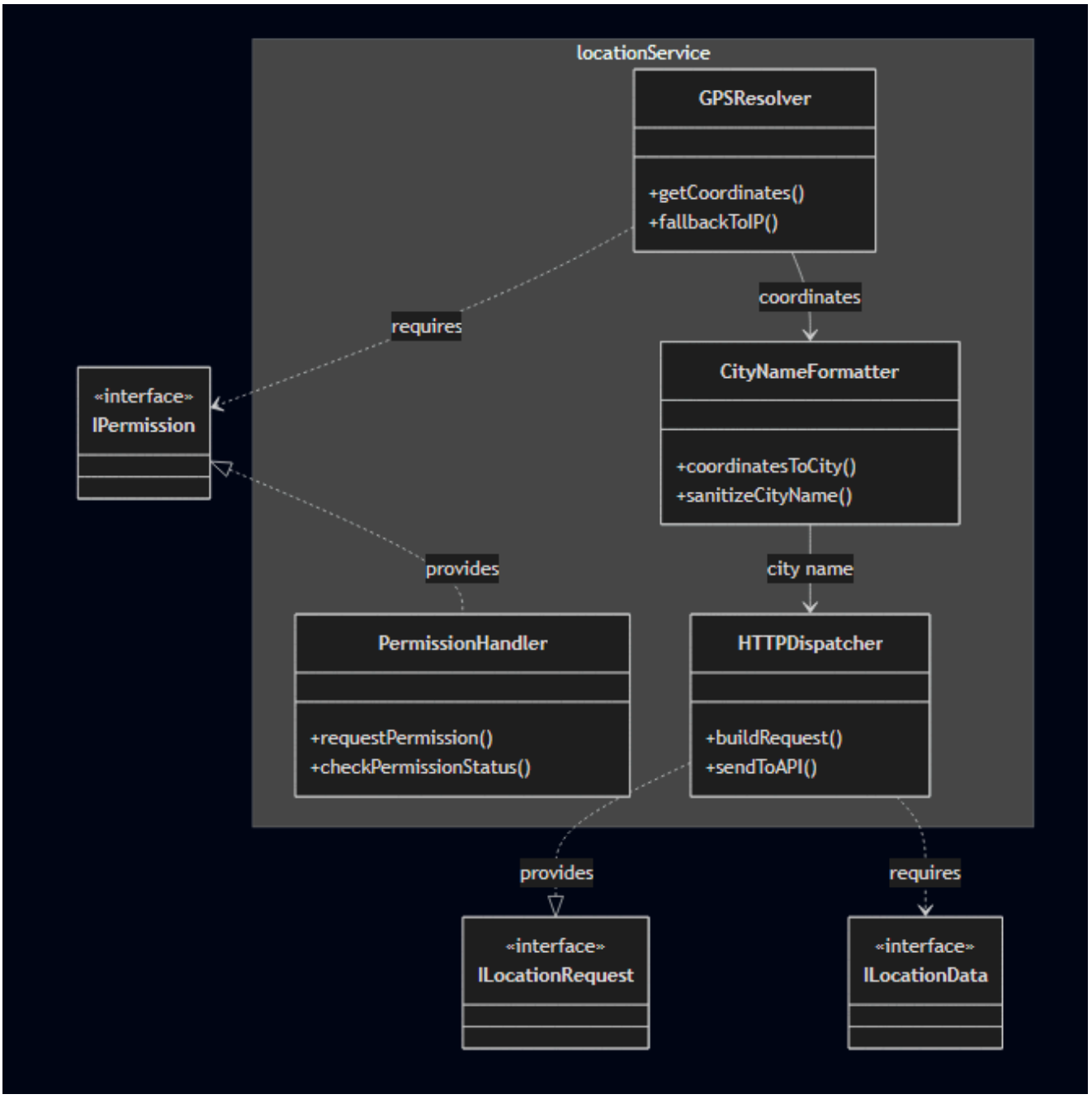
```
}



class locationService{



}

class AirVisualDatabase{



}

class display{

}



locationService --> APIAPP : user's location and air cond. data request

APIAPP --> APIDatabase : Location and data request

APIDatabase --> AirVisualDatabase : location and data request

AirVisualDatabase --> APIDatabase : air cond.

APIDatabase --> APIAPP : air cond.

APIAPP --> display : aircond.
```



locationService

```
classDiagram
    namespace locationService {
        class PermissionHandler {
            +requestPermission()
            +checkPermissionStatus()
        }
        class GPSResolver {
            +getCoordinates()
            +fallbackToIP()
        }
        class CityNameFormatter {
            +coordinatesToCity()
            +sanitizeCityName()
        }
        class HTTPDispatcher {
            +buildRequest()
```

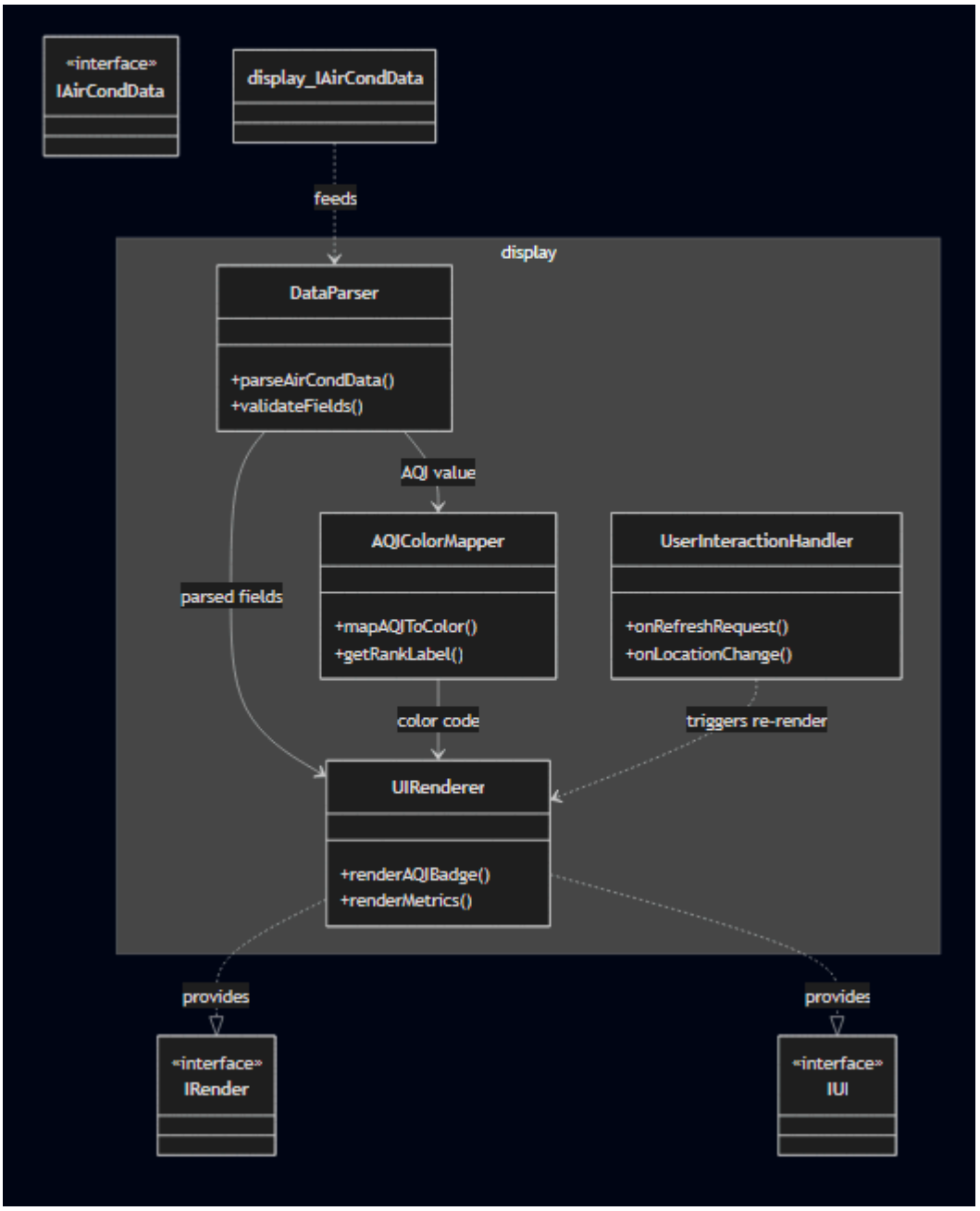
```
        +sendToAPI()
    }
}
class IPermission {
    <<interface>>
}
class ILocationRequest {
    <<interface>>
}
class ILocationData {
    <<interface>>
}

IPermission <|.. PermissionHandler : provides
GPSResolver ..> IPermission : requires
GPSResolver --> CityNameFormatter : coordinates
CityNameFormatter --> HTTPDispatcher : city name
HTTPDispatcher ..|> ILocationRequest : provides
HTTPDispatcher ..> ILocationData : requires
```



display

```
classDiagram
    namespace display {
        class AQIColorMapper {
            +mapAQIToColor()
            +getRankLabel()
        }
        class DataParser {
            +parseAirCondData()
            +validateFields()
        }
        class UIRenderer {
            +renderAQIBadge()
            +renderMetrics()
        }
        class UserInteractionHandler {
```

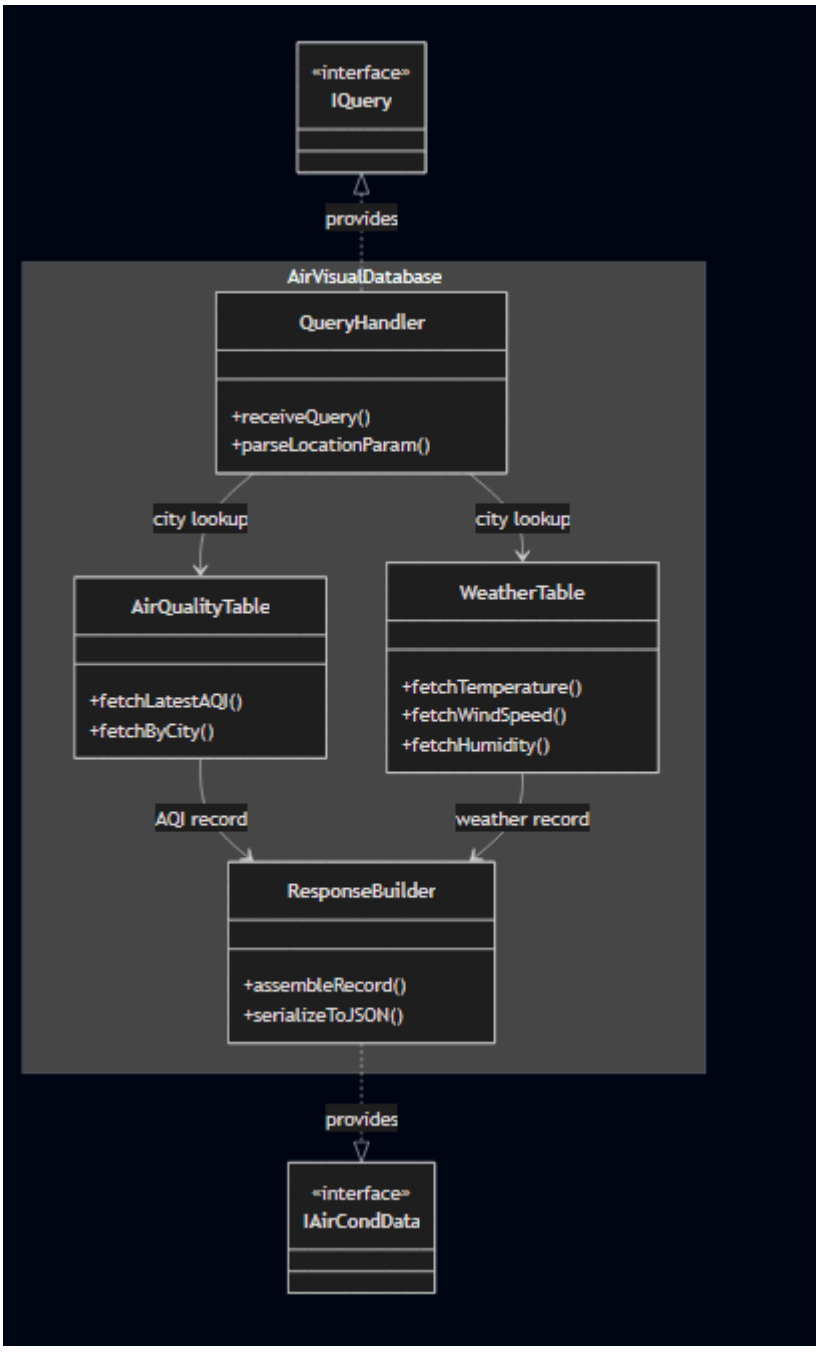
```
        +onRefreshRequest()
        +onLocationChange()
    }
}
class IAirCondData {
    <<interface>>
}
class IUI {
    <<interface>>
}
class IRender {
    <<interface>>
}

display_IAirCondData ..> DataParser : feeds
DataParser --> AQIColorMapper : AQI value
DataParser --> UIRenderer : parsed fields
AQIColorMapper --> UIRenderer : color code
UIRenderer ..|> IRender : provides
UIRenderer ..|> IUI : provides
UserInteractionHandler ..> UIRenderer : triggers re-render
```



database

```
classDiagram
    namespace AirVisualDatabase {
        class QueryHandler {
            +receiveQuery()
            +parseLocationParam()
        }
        class AirQualityTable {
            +fetchLatestAQI()
            +fetchByCity()
        }
        class WeatherTable {
            +fetchTemperature()
            +fetchWindSpeed()
            +fetchHumidity()
        }
```

```
        class ResponseBuilder {
            +assembleRecord()
            +serializeToJSON()
        }
    }
    class IQuery {
        <<interface>>
    }
    class IAirCondData {
        <<interface>>
    }

    IQuery <|.. QueryHandler : provides
    QueryHandler --> AirQualityTable : city lookup
    QueryHandler --> WeatherTable : city lookup
    AirQualityTable --> ResponseBuilder : AQI record
    WeatherTable --> ResponseBuilder : weather record
    ResponseBuilder ..|> IAirCondData : provides
```


«interface»
IQuery
provides
AirVisualDatabase
QueryHandler
+receiveQuery()
+parseLocationParam()
city lookup
city lookup
AirQualityTable
+fetchLatestAQI()
+fetchByCity()
WeatherTable
+fetchTemperature()
+fetchWindSpeed()
+fetchHumidity()
AQI record
weather record
ResponseBuilder
+assembleRecord()
+serializeToJSON()
provides
«interface»
IAirCondData